
task-app 30日間カリキュラム

Day 09

プロジェクト一覧画面を作ろう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	3
なぜこれを作るのか	4
実装ステップ一覧	8
Step 0 : プロジェクト一覧 API を自分の手で書く(読む目安: 15分).....	9
Step 1 : ページの骨組みを作る(読む目安: 3分).....	17
Step 2 : tRPC でデータを取得する(読む目安: 5分)	23
Step 3 : 読み込み中と取得失敗の表示を作る(読む目安: 5分)	25
Step 4 : カード表示用の import を追加する(読む目安: 3分).....	29
Step 5 : イベントハンドラーを準備する(読む目安: 4分)	31
Step 6 : プロジェクトカードをグリッド表示する(読む目安: 5分)	33
Step 7 : 空状態のメッセージを追加する(読む目安: 3分)	37
Step 8 : ヘッダー部品の import と状態を準備する(読む目安: 5分).....	41
Step 9 : ページ全体を組み立てる(読む目安: 5分).....	44
Step 10 : 動作確認(読む目安: 3分).....	52
完成コード全体	60
今日のまとめ	78
つまずきポイント	79
今日学んだ用語.....	81
応用課題: 最初に表示する絞り込みを変える.....	82
理解チェック	82
次回予告	83
Day 09 完成形コード(参照用).....	84
次に読むもの	85
奥付	86

前回の振り返り

Day 08 ではサイドバーにユーザー情報ウィジェットとログアウト確認ダイアログを実装し、認証ガードによる未ログイン時のリダイレクトも体験しました。認証まわりが完成したので今日からアプリの中核機能であるプロジェクト管理に取り組みます。

今日のゴール

tRPC の `useQuery` を使ってサーバーからプロジェクトデータを取得し、カード形式で一覧表示します。グリッドレイアウトでレスポンス対応も実装します。

スクリーンショット: 完成イメージ(プロジェクト一覧画面)

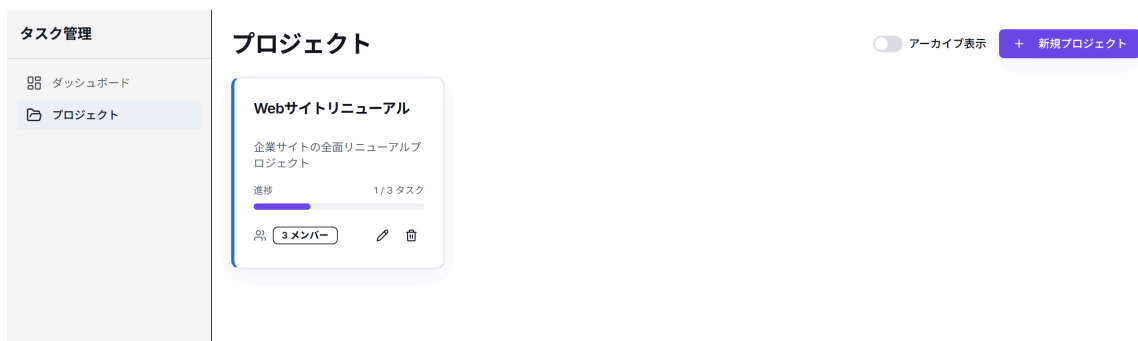


図1: サイドバーに「ダッシュボード」と「プロジェクト」が並ぶプロジェクト一覧。カードに色帯・進捗・メンバー数が並ぶ

画像は今日の完成状態です。サイドバーには「ダッシュボード」と、今日追加する「プロジェクト」の2項目が表示されます。「マイタスク」はページを作る Day 17 で追加します。

なぜこれを作るのか

このアプリではタスクとメンバーが「プロジェクト」にぶら下がります。だから中身づくりの最初はプロジェクトを一覧で見渡せてそこから各プロジェクトへ入っていける入口を用意します。

例え話: プロジェクト一覧は「本棚」です。本棚に並んだ本(プロジェクト)を一目で見渡し、1冊ずつ手に取って詳細を確認できます。まずは本棚を作りましょう。

データ取得の流れ

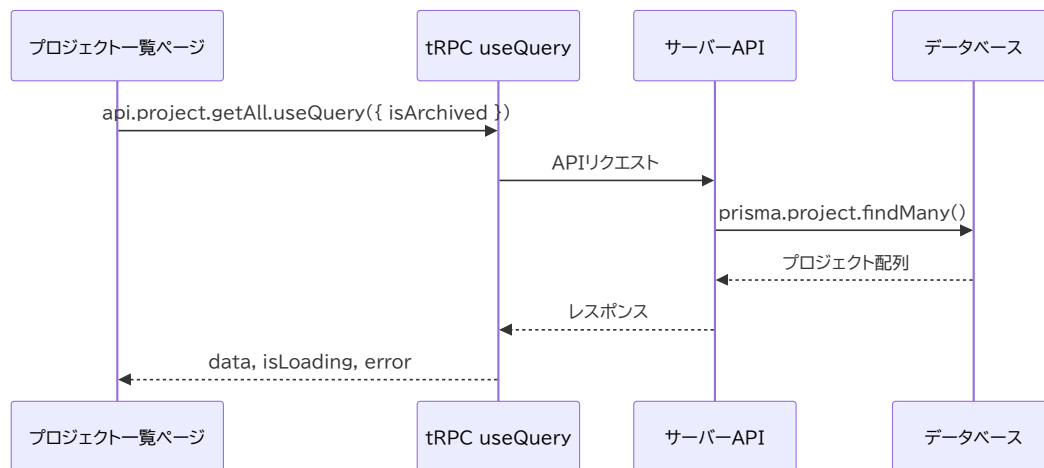


図 2: データ取得の流れ

この図で目を留めてほしいのはいちばん下の矢印です。ページが受け取るのは `data` だけでなく、`isLoading` と `error` も一緒です。リクエストはページを出てサーバーとデータベースまで届き、そこから折り返してきます。だから画面が現れた瞬間、`data` はまだ `undefined` のままです。Step 3 で読み込み中の表示に切り替えるのはこの空白の時間があるからです。データベースにつなぐのはサーバー側だけです。ページ自身は `prisma` を知りません。Day 07 で作った認証 API も同じ道を通ります。

やること / やらないこと

やること	やらないこと
<code>useQuery</code> でデータ取得	データの作成・編集(Day 10-11)
グリッドレイアウトで一覧表示	詳細ページの実装
ローディング・エラー表示	メンバー管理(Day 12)
ProjectCard コンポーネントの表示	カードのデザインをゼロから作る

Day 09: プロジェクト一覧画面を作ろう

今日触るファイル

```
src/  
├─ app/  
│   └─ project/  
│       └─ page.tsx      ← メイン(新規作成)  
├─ server/  
│   └─ api/  
│       ├── root.ts      ← 既存(project を登録する)  
│       └─ routers/  
│           └─ project.ts ← 一覧 API(新規作成)  
├─ component/  
│   ├── project/  
│   │   └─ project-card.tsx ← 今日の表示で使うカード  
│   ├── layout/  
│   │   └─ app-layout.tsx  ← 既存(利用する)  
│   └─ ui/  
│       └─ loading-spinner.tsx ← 既存(利用する)  
└─ lib/  
    └─ constant/  
        └─ status.ts      ← 既存(利用する)
```

今日は `src/app/project/page.tsx` をゼロから書いていく方式で進めます。すでにファイルがある場合は一旦中身を空にしてステップごとにコピペしてください。

新しく学ぶ概念

概念	読み方	役割	例え
useQuery (Day 08 の 復習)	ユーズ・クエリ	サーバーから データを取得する フック	図書館の検索端末。リクエ ストすると結果が返って くる
グリッドレイア ウト	—	要素を格子状に並 べる CSS	本棚の棚板。横に何冊並 べるかを画面幅で変える
Suspense	サスペンス	コンポーネントの 準備が終わるまで 代替りの画面を表 示する React の 仕組み	レストランの「ただいま準 備中」の看板

Suspense の役割を理解する

用語	何をするか	今日の使い方
<code>Suspense</code>	子コンポーネントが準備 完了するまで <code>fallback</code> を表示する	ページ全体のローディング ガード
<code>fallback</code>	準備中に表示する UI	<code><PageLoadingSpinner /></code>
<code>PageLoadingSpinner</code>	スピナー付きのローディ ング表示	<code>Suspense</code> の <code>fallback</code> や <code>isLoading</code> 時に使う

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 0	プロジェクト一覧 API (getAll)を自分で書く	15分
Step 1	ページの骨組みとサイドバー導線を作る	5分
Step 2	tRPCでデータを取得する	5分
Step 3	読み込み中と取得失敗の表示を作る	5分
Step 4	カード表示用のimportを追加する	3分
Step 5	イベントハンドラーを準備する	4分
Step 6	プロジェクトカードをグリッド表示する	5分
Step 7	空状態のメッセージを追加する	3分
Step 8	ヘッダー部品のimportと状態を準備する	5分
Step 9	ページ全体を組み立てる	5分
Step 10	動作確認	3分

読む時間の合計(仮): 約 58 分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 0 : プロジェクト一覧 API を自分の手で書く(読む目安: 15 分)

ゴール: プロジェクト一覧を返す `getAll` を自分で書き、`root.ts` に登録して画面から `api.project.getAll` を呼べる状態にします。

一覧画面にはサーバーが持っているプロジェクトを画面まで運んでくる入口が必要です。その入口を、今日は自分の手で1つ作ります。Day 07 で認証 API(`auth.ts`)を作り直したのと同じ流れです。

tRPC の手続きは3つの部品でできている

これから書く `getAll` に限らず、tRPC の手続き(procedure)はいつも同じ3部品の組み合わせです。この形を先に頭へ入れておくと Day 10 以降で `create` や `update` を書くときも、同じ型に当てはめるだけで済みます。

部品	役割	<code>getAll</code> での中身
入力(input)	クライアントから何を受け取るか。 <code>z</code> で形を検証する	アーカイブを含めるか、などの絞り込み条件
処理(query / mutation)	受け取った条件で DB に問い合わせる	Prisma でプロジェクトを検索する
戻り値(return)	画面に返すデータ	プロジェクトの配列

Day 09: プロジェクト一覧画面を作ろう

今日はこの3部品のうち、一覧取得の `getAll` だけを書きます。`create` や `update` はそれを実際に使う Day 10 以降で1つずつ足していきます。1日で全部書く必要はありません。

0-1. まず import から

`src/server/api/routers/project.ts` を新規作成し、先頭に import を書きます。

```
// filepath: src/server/api/routers/project.ts
import type { Prisma } from '@prisma/client';
import { TRPCError } from '@trpc/server';
import { z } from 'zod';
import { USER_ROLE } from '@/lib/constant/roles';
import { prisma } from '@/lib/prisma';
import { createTRPCRouter, protectedProcedure } from
'../trpc';
import { USER_SELECT } from '../_helpers/select';
```

import は「これから使う道具を最初に並べておく」宣言です。先頭の

`import type` (型だけを取り込み、実行時には消える書き方)で `Prisma` を入れているのは値としてではなく `where` の型注釈にだけ使うためです。

`protectedProcedure` はログイン済みの人だけが呼べる手続きを作る道具、`z` は入力の形を検証する道具、`prisma` は DB に問い合わせる道具です。

`USER_SELECT` は Day 07 で作った `_helpers/select.ts` にある「ユーザーのどの項目を返すか」の指定です。パスワードなど返してはいけない項目を毎回書かずに済むよう、まとめてあります。

0-2. 手続きの骨組みと入力を書く

`getAll` の骨組みを書きます。`protectedProcedure` で始めるとログインしていない人がこの API を呼んだときに自動で弾かれます。`.input(...)` では受け取る条件を定義します。

```
// filepath: src/server/api/routers/project.ts(続き)
export const projectRouter = createTRPCRouter({
  getAll: protectedProcedure
    .input(
      z
        .object({
          userId: z.string().cuid().optional(),
          isArchived: z.boolean().optional(),
        })
        .optional(),
    )
    .query(async ({ ctx, input }) => {
      const where: Prisma.ProjectWhereInput = {};
    });
```

`userId` と `isArchived` の各項目に `.optional()` が付いているのはその項目だけ省略してよいという意味です。いちばん外側にも `.optional()` があるので条件オブジェクトごと渡さずに呼ぶこともできます。何も渡さなければ「ログイン中の自分のプロジェクトを全部」返します。`.query(...)` の中の `ctx` にはログイン中のユーザー情報が入り、`input` には今定義した条件が入ってきます。`where` はこのあと組み立てる「どのプロジェクトを探すか」の検索条件を入れておく変数です。

0-3. ここが一番のヤマ場(誰の一覧を返すかを守る)

ここが `getAll` で最も気をつける部分です。もし他人の `userId` を渡すだけで他人のプロジェクトを覗いてしまうと見えてはいけないうものが見えてしまいます。そこで「自分以外の `userId` を指定できるのは管理者だけ」という制限を入れます。

```
// filepath: src/server/api/routers/project.ts(続き)
    if (input?.userId && input.userId !==
ctx.session.userId) {
    if (ctx.session.role !== USER_ROLE.ADMIN) {
        throw new TRPCError({
            code: 'FORBIDDEN',
            message: '管理者権限が必要です',
        });
    }
}
```

`input.userId !== ctx.session.userId` は「渡された `userId` が自分自身ではない」という意味です。手前の `input?.userId` にある `?.` (オプションチェーン、途中の値が無ければそこで止めて `undefined` を返す書き方)は条件オブジェクトごと省略して呼ばれても落ちないようにするためです。それが管理者以外だったときに `TRPCError` を `throw` すると処理はここで止まり、画面には権限がないというエラーが返ります。`throw` は「これ以上は進めない」とその場で処理を打ち切る命令です。

0-4. 検索条件を組み立てる

弾く条件を通過したら実際に「どのプロジェクトを探すか」を `where` に組み立てます。

```
// filepath: src/server/api/routers/project.ts(続き)

if (!input?.userId) {
  where.members = {
    some: { userId: ctx.session.userId },
  };
} else {
  where.members = {
    some: { userId: input.userId },
  };
}

if (input?.isArchived !== undefined) {
  where.isArchived = input.isArchived;
}
```

`members.some` は「メンバーにこのユーザーが含まれるプロジェクト」という条件です。`some` は Prisma で「関連の中に条件を満たすものが1つでもあれば対象にする」という書き方です。`userId` を渡さなければ自分がメンバーのものを、渡せば指定した人がメンバーのものを探します。自分のIDは管理者以外でも指定できます。他人のIDを指定する場合だけ、先ほどの管理者チェックが必要です。`isArchived` は指定されたときだけ条件に足すので未指定のときはアーカイブ済みも含めた全部が対象になります。

0-5. Prisma でプロジェクトを取得して返す

組み立てた `where` を使って Prisma で一覧を取得します。画面はメンバー数とタスク進捗を表示するので関連するメンバーとタスクも `include` で一緒に取ってきます。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/server/api/routers/project.ts(続き)
return await prisma.project.findMany({
  where,
  include: {
    members: {
      include: {
        user: {
          select: USER_SELECT,
        },
      },
    },
    tasks: {
      select: {
        id: true,
        status: true,
      },
    },
  },
  orderBy: { createdAt: 'desc' },
});
```

このコードで一番分かりにくいのは `include` が3階層も入れ子になっているところです。ここは1つずつ分解すれば読めます。まず Prisma では `include` と `select` の役割が違うことを押さえます。

キーワード	役割	このコードでの使い方
<code>include</code>	関連するデータも一緒に 取ってくる	プロジェクトに紐づくメン バーとタスクを取る
<code>select</code>	取ってきたデータのうち、 返す項目だけに絞る	ユーザーは名前やアイコ ンだけ、タスクは <code>id</code> と <code>status</code> だけ

入れ子が3階層になるのは関連をたどってさらにその先の関連を取るからです。プロジェクト → (`include: members`) そのメンバー → (`include: user`) メンバーが誰なのかを表すユーザー、と2回たどるので `include` の中に `include` が入ります。一番内側のユーザーは `select: USER_SELECT` で必要な項目だけに絞り、パスワードなどは返しません。`tasks` は進捗の集計にしか使わないので `select` で `id` と `status` の2つだけ取ります。全項目を取ると通信が重くなるからです。

こうしてメンバーとタスクを一緒に取っておくと画面側はメンバー数やタスクの進捗を追加の通信なしで表示できます。ただしカードが使うのは件数だけなのにタスクの行そのものを全部運んでいます。プロジェクトごとにタスクが数百件あればその数百行が毎回届きます。件数だけが欲しい場面ではPrismaの`_count`を使うとDB側で数えてから数字だけを返せます。今日は`include`の形を覚えることを優先しています。`orderBy: { createdAt: 'desc' }`は作成日の新しい順に並べる指定です。最後の`}},`で`getAll`を閉じ、`});`で`projectRouter`全体を閉じます。

Day 09: プロジェクト一覧画面を作ろう

0-6. root.ts に project ルーターを登録する

`getAll` を書いただけではまだ画面から呼べません。作った `projectRouter` を `root.ts` に登録して初めて `api.project.getAll` という呼び名が生まれます。Day 07 で `auth` を登録したのと同じ形です。

```
// filepath: src/server/api/root.ts
import { authRouter } from './routers/auth';
import { projectRouter } from './routers/project';
import { createCallerFactory, createTRPCRouter } from
'./trpc';

export const appRouter = createTRPCRouter({
  auth: authRouter,
  project: projectRouter,
});

export type AppRouter = typeof appRouter;

export const createCaller = createCallerFactory(
appRouter);
```

`appRouter` に `project: projectRouter` を足したことで、フロント側の `api.project.getAll` がこの手続きにつながります。今の `root.ts` には Day 07 の `auth` と今日の `project` の2つだけが並びます。`comment` や `task` などはそれを使う Day で1つずつ足していきます。

次の Day 10 で書く `create` も、今日と同じ3部品(入力・処理・戻り値)でできています。違うのは処理が「探す(`.query`)」ではなく「作る(`.mutation`)」になる点です。だから今日 `getAll` の形をつかんでおけば Day 10 は同じ骨組みに中身を入れ替えるだけになります。

確認ポイント:

- `src/server/api/routers/project.ts` に `getAll` を書き、`}},` と `});` まで閉じました。
- `root.ts` に `projectRouter` の import と `project: projectRouter` の2行を追加しました。
- ここまでのコードの綴りと括弧の対応を確認しました(ファイル全体の型検査は完成後に行います)。

Step 1 : ページの骨組みを作る(読む目安: 3分)

ゴール: プロジェクト一覧ページの基本構造を作ります。

実装:

`src/app/project/page.tsx` を新規作成します。`src/app/` の下に `project` フォルダを作り、その中に `page.tsx` を置いてください。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
'use client';

import { Suspense } from 'react';
import { AppLayout }
  from '@component/layout/app-layout';
import { PageLoadingSpinner }
  from '@component/ui/loading-spinner';
```

先頭の `'use client'` (このファイルをブラウザ側で動かす宣言) はあとで `useState` などのフックをこのページで使うために必要です。続く取り込みはページを組み立てる部品です。`Suspense` は準備が終わるまで仮の表示に差し替える仕組み、`AppLayout` はサイドバー付きの共通枠、`PageLoadingSpinner` は読み込み中に出すスピナーです。

確認ポイント:

- ファイルを保存しました。
- `'use client'` がファイル先頭にあります。

続けて同じファイルの末尾に以下を追加します。

```
// filepath: src/app/project/page.tsx
function ProjectPageContent() {
  return (
    <AppLayout>
      <div className="flex flex-col gap-6">
        <h1 className="text-3xl font-bold tracking-tight">
          プロジェクト
        </h1>
      </div>
    </AppLayout>
  );
}
```

ここではまず中身のない骨組みを作ります。`AppLayout` で全体を囲むとこのページにもサイドバーやヘッダーが付きます。内側の `<div>` は縦並びの入れ物で、いまは見出しの `<h1>` だけを置き、カード一覧はあとのステップで足していきます。

確認ポイント:

- `AppLayout` でページ全体を囲んでいます。

最後にページのエクスポートを追加します。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
export default function ProjectPage() {
  return (
    <Suspense
      fallback={<PageLoadingSpinner />}>
      <ProjectPageContent />
    </Suspense>
  );
}
```

この `ProjectPage` が実際に画面へ表示されるコンポーネントです。本体の `ProjectPageContent` を `Suspense` で包むのは Day 05 のログイン画面と同じ形をここでも作っておくためです。このページがあとで URL の値を読むようになったとき包む場所を作り直さずに済みます。

確認ポイント:

- `npm run dev` でエラーが出ていません。
- ブラウザで `/project` にアクセスして「プロジェクト」と表示されます。
- サイドバーが表示されています。

スクリーンショット: 下の画像は Step 9 まで書き終えた完成後の画面です。赤枠の中がこの Step で出る見出しです。いまの自分の画面には右上のアーカイブ表示スイッチと新規プロジェクトボタン、そして下のカードがまだ出ていません。カードは Step 6、ヘッダーの2つは Step 9 で足します。

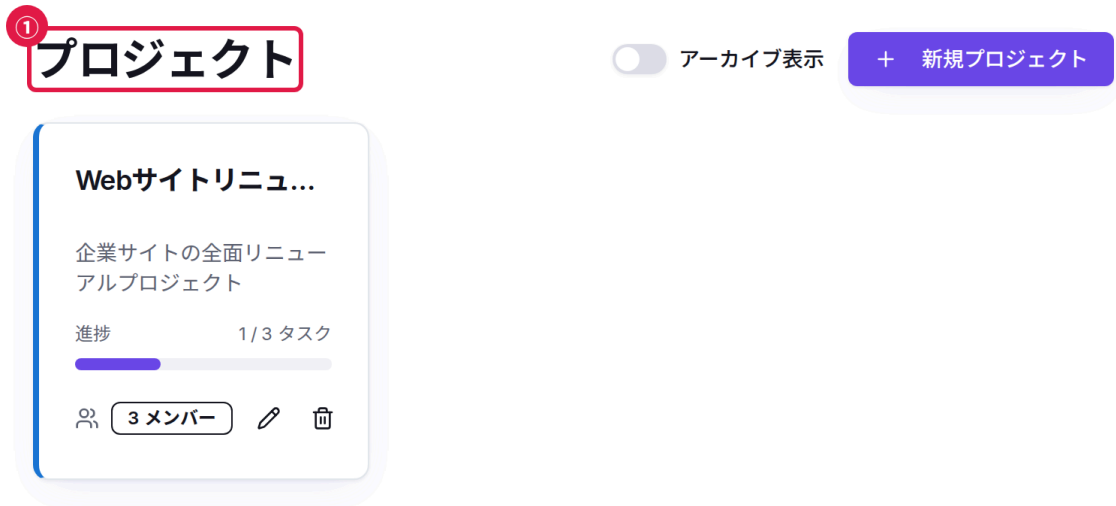


図3: 完成後のプロジェクト一覧。赤枠の中に「プロジェクト」の見出しが出ている

`Suspense` は中身が宙づりになったときだけ `fallback` の `PageLoadingSpinner` を代わりに出します。今日の `ProjectPageContent` は宙づりにならないのでこの `fallback` は実際には出ません。読み込み中の見た目は次の Step で足す `projectsLoading` の分岐が作ります。

`/project` のページが表示できたので、サイドバーにも入口を追加します。ページより先にリンクを置かないのは、作業途中でも押せるリンクを必ず有効に保つためです。

`src/component/layout/app-layout.tsx` の `lucide-react` の import を次の形にします。

```
// filepath: src/component/layout/app-layout.tsx
import { FolderOpen, LayoutDashboard, Logout } from
  'lucide-react';
```

Day 09: プロジェクト一覧画面を作ろう

`FolderOpen` だけが今日の追加です。`LayoutDashboard` と `LogOut` は Day 08 から使っているので残します。ここで3つをまとめて示すと、既存の名前を消さずにどこへ足すかを判断できます。

続けて `menuItems` を次の2項目に置き換えます。

```
// filepath: src/component/layout/app-layout.tsx
const menuItems: MenuItem[] = [
  {
    text: 'ダッシュボード',
    icon: <LayoutDashboard className="h-5 w-5" />,
    path: '/dashboard',
  },
  {
    text: 'プロジェクト',
    icon: <FolderOpen className="h-5 w-5" />,
    path: '/project',
  },
];
```

`FolderOpen` はプロジェクト項目のアイコンです。`path` は先ほど作った `src/app/project/page.tsx` と同じ `/project` にします。フォルダ名と綴りが違うと、メニューは表示されても押した先が404になります。

確認ポイント:

- サイドバーに「プロジェクト」が追加されました。
- 「プロジェクト」を押すと `/project` が開きます。

Step 2 : tRPCでデータを取得する(読む目安: 5分)

ゴール: `useQuery` でプロジェクト一覧をサーバーから取得します。

実装:

`src/app/project/page.tsx` の import 群に以下を追加します。

```
// filepath: src/app/project/page.tsx
// import群に追加(ファイル先頭のimportの後ろ)
import { api } from '@trpc/react';
import { useState } from 'react';
```

`api` はサーバー側に置いたルーターへつながる入口です。ここから呼べる関数の名前も、渡せる引数の形も、サーバー側の定義から自動で決まります。だから `getAll` を打ち間違えるとブラウザで開く前にエディタが赤い波線で教えてくれます。手書きの `fetch` ではURL を間違えても実行するまで気づけません。

`useState` はアーカイブ表示を出すかどうかの選択を覚えておくために使います。ただの変数に入れると画面が描き直されたときに値が初期状態へ戻ります。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。

`ProjectPageContent` 関数の先頭(`return` の前)に以下を追加します。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
// ProjectPageContent内、returnの前に追加
const [showArchived, setShowArchived] =
  useState(false);

const {
  data: projects,
  isLoading: projectsLoading,
  error: projectsError,
} = api.project.getAll.useQuery({
  isArchived: showArchived,
});
```

`useQuery` に `isArchived: showArchived` を渡しています。スイッチを切り替えると `showArchived` の値が変わり、tRPC はそれを別の問い合わせとして扱って取り直します。取得しなおす関数を自分で呼ぶ必要はありません。切り替えのたびに再取得を書く形にすると書き忘れた場所だけ古い一覧が残ります。

`projects` は返事が届くまで `undefined` です。`data: projects = []` と書くとこの間は空配列を使えます。ただし値だけでは読み込み中と0件を見分けられません。`isLoading` で読み込み中を分け、`error` で取得失敗を分けます。

確認ポイント:

- `npm run dev` でエラーが出ていません。
- ブラウザの開発者ツール → コンソールにエラーが出ていません。

useQueryの返回值

返回值	説明
<code>data</code>	取得したデータ(読み込み前は <code>undefined</code>)
<code>isLoading</code>	初回読み込み中かどうか
<code>error</code>	tRPCのエラー情報(正常時は <code>null</code>)
<code>isRefetching</code>	再取得中かどうか

`useQuery` はページ表示時に自動でAPIを呼びます。手動で `fetch` を書く必要はありません。

Step 3 : 読み込み中と取得失敗の表示を作る(読む目安: 5分)

ゴール: データの読み込み中と取得失敗を、0件の状態と分けて表示します。

実装:

`ProjectPageContent` 内の `return` の直前に追加します。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
// ProjectPageContent内、returnの前に追加
const projectAccessDenied =
  projectsError?.data?.code === 'UNAUTHORIZED' ||
  projectsError?.data?.code === 'FORBIDDEN';

if (projectsLoading) {
  return (
    <AppLayout>
      <PageLoadingSpinner />
    </AppLayout>
  );
}
```

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。
- `projectsLoading` が `true` の間はスピナーを出します。
- 最初の取得失敗と認証・認可の失敗ではカードを表示しません。

スクリーンショット: 読み込んでいる間は本文の場所にスピナーだけが出ます。

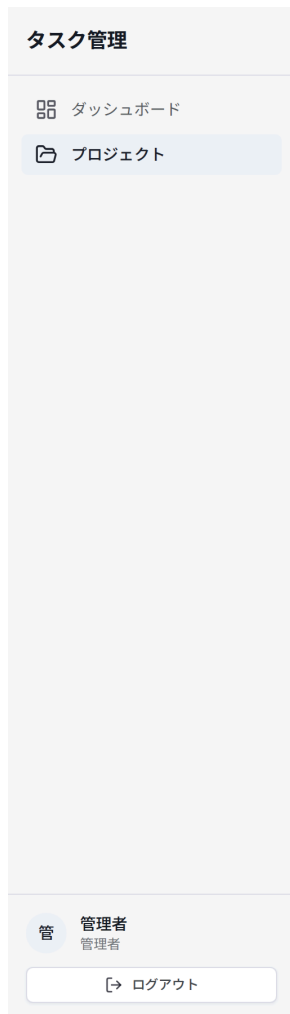


図4: 読み込み中のプロジェクト一覧。左のサイドバーは出たままで右の本文の場所に丸いスピナーだけが回っている

スピナーも `AppLayout` で囲みます。`AppLayout` を書いているのはこの下の `return` の中なのでここで早く `return` すると読み込み中だけサイドバーとログイン確認が画面から消えます。

次に、一度も一覧を取得できなかった場合の表示を、読み込み中の分岐の後ろに追加します。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
// 読み込み中の分岐の後ろに追加
if (projectsError && (!projects || projectAccessDenied)) {
  return (
    <AppLayout>
      <p role="alert">
        {projectAccessDenied
          ? 'この一覧を表示する権限がありません。'
          : 'プロジェクトを読み込めませんでした。'}
      </p>
    </AppLayout>
  );
}
```

UNAUTHORIZED はログインを確認できない失敗、**FORBIDDEN** は操作の権限がない失敗です。どちらかなら前回の一覧がキャッシュに残っていても表示しません。通信の一時的な失敗で前回の一覧がある場合は、そのカードを残します。

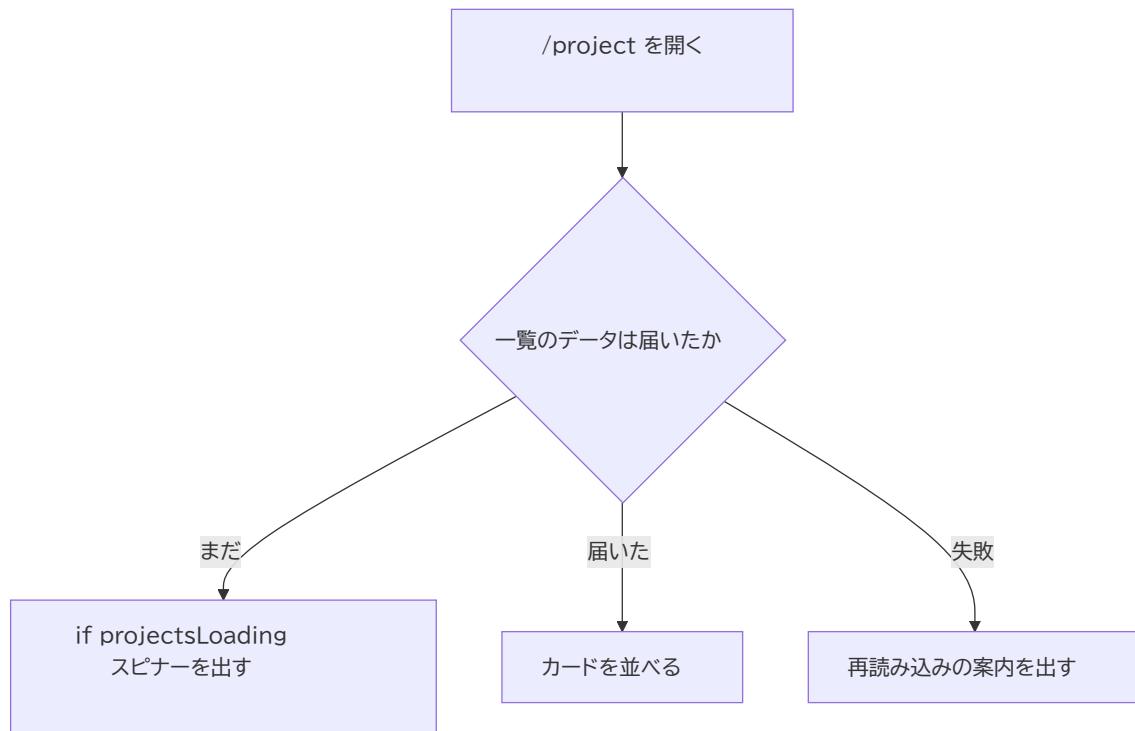


図5: Step 3 : 読み込み中と取得失敗の表示を作る

読み込み中の表示を受け持つのは `projectsLoading` の分岐です。取得失敗は次の `projectsError` の分岐が受け持ちます。Step 1 で置いた `Suspense` の `fallback` はいまの `ProjectPageContent` では出番がありません。

`useQuery` はデータを待つ間も普通に描画を返すので中身が宙づり(suspend)にならないからです。`Suspense` が仕事をするのはあとから読み込む部品を包んだときや、URL の値が確定するまで待つ `useSearchParams` を包んだときで、Day 05 のログイン画面がその形でした。

Step 4 : カード表示用のimportを追加する(読む目安: 3分)

ゴール: プロジェクトカードと定数のimportを追加します。

実装:

Day 09: プロジェクト一覧画面を作ろう

`src/app/project/page.tsx` の import 群に以下を追加します。

```
// filepath: src/app/project/page.tsx
// import群に追加
import { ProjectCard }
  from '@component/project/project-card';
import { TASK_STATUS }
  from '@lib/constant/status';
```

`ProjectCard` は色帯・メンバー数・進捗バーまで作り込んであるカード部品です。今日はこれを自分で書きません。Step 0 の `getAll` が返してきたデータを流し込むことに集中します。`TASK_STATUS` のほうはそのデータに含まれる `status` と突き合わせるための定数です。完了を表す文字列が `TASK_STATUS.DONE` です。Step 6 で完了件数を数えるときに使います。この2行を取り込まないまま Step 6 へ進むとどちらの名前も「見つかりません」という型エラーになります。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。

ProjectCardのprops

prop	型	説明
<code>id</code>	<code>string</code>	プロジェクトID
<code>name</code>	<code>string</code>	プロジェクト名
<code>description</code>	<code>string null</code>	説明文(任意)

prop	型	説明
<code>color</code>	<code>string</code>	カラーコード(例: <code>#1976d2</code>)
<code>memberCount</code>	<code>number</code>	メンバー数
<code>taskStats</code>	<code>{total, done}</code>	タスク進捗
<code>onEdit</code>	<code>(id: string) => void</code>	任意。省略すると編集ボタンを表示しません
<code>onDelete</code>	<code>(id: string) => void</code>	任意。省略すると削除ボタンを表示しません
<code>onClick</code>	<code>(id: string) => void</code>	任意。渡すとカードとプロジェクト名から開けます
<code>isArchived</code>	<code>boolean</code>	アーカイブ済みか

`onClick` を渡したカードでは、カード全体をマウスで押せます。プロジェクト名はボタンになるため、`Tab` キーで選び、`Enter` キーでも開けます。`onClick` を省略した場合は、名前を普通の文字として表示します。

Step 5 : イベントハンドラーを準備する(読む目安: 4分)

ゴール: カードのボタンに渡すハンドラー関数を準備します。

実装:

`ProjectPageContent` 内、`if (projectsLoading)` の前に追加します。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
// ProjectPageContent内に追加
const handleEdit = (projectId: string) => {
  void projectId;
};
const handleDelete = (projectId: string) => {
  void projectId;
};
const handleProjectClick = (id: string) => {
  void id;
};
```

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。
- 型エラーが出ていません。

仮実装について: `handleEdit` / `handleDelete` /

`handleProjectClick` は今の時点では何もしません。Day 11 で編集ダイアログ・削除確認・詳細表示を実装するので今日はカードに渡すための「受け皿」だけ作っておきます。ボタンをクリックしても今は何も起きませんがそれで正常です。

`void projectId` は「この引数を意図的に使わない」ことを TypeScript に伝える書き方です。 `_` プレフィックスの代わりに使えるテクニックです。

Step 6 : プロジェクトカードをグリッド表示する(読む目安: 5分)

ゴール: プロジェクトをカード形式でグリッド表示します。

実装:

`ProjectPageContent` の `return` 内にある `<h1>` タグの後に、以下のグリッド表示を追加します。これは完成形のコードブロックです。

```
{/* filepath: src/app/project/page.tsx */}
{/* return内、</h1>の後にグリッド開始 */}
<div className="grid gap-6 sm:grid-cols-2
  lg:grid-cols-3 xl:grid-cols-4">
  {projects?.map((project) => {
    // キャンセル済みは進捗の母数に含めない
    let taskCount = 0;
    let doneCount = 0;
    for (const t of project.tasks ?? []) {
      if (t.status === TASK_STATUS.CANCELLED) continue;
      taskCount++;
      if (t.status === TASK_STATUS.DONE) doneCount++;
    }
  })}
```

`grid` のクラスは画面の幅に応じた列数を指定します。中の `.map()` はプロジェクトの配列を1件ずつカードに変換する繰り返しです。

件数を数える部分では初めて出る書き方が5つあります。`let` は、あとから中身を増やせる変数を作る宣言です。`for (const t of 配列)` は配列の中身を1件ずつ `t` に入れて繰り返します。`?? []` は左が `null` や `undefined` のときだけ空の

Day 09: プロジェクト一覧画面を作ろう

配列を使う書き方です。タスクが0件でもエラーにせず数えられます。`continue`はその1件を飛ばして次へ進む合図です。キャンセル済みを分母に含めると達成率が実際より低く出るため、ここで飛ばします。`++` は数を1つ増やします。

確認ポイント:

- グリッドの開始タグとmap処理を追加しました。

map の中で `ProjectCard` を返します。上のコードブロックの続きです。

```
// filepath: src/app/project/page.tsx
// map内のreturn(上の続き)
    return (
      <ProjectCard
        key={project.id}
        id={project.id}
        name={project.name}
        description={project.description}
        color={project.color}
        memberCount={
          project.members?.length ?? 0}
        taskStats={{
          total: taskCount,
          done: doneCount,
        }}
        onEdit={handleEdit}
        onDelete={handleDelete}
        onClick={handleProjectClick}
        isArchived={project.isArchived}
      />
    );
  })}
</div>
```

map の中で、各プロジェクトの値を `ProjectCard` に渡してカードを表示します。
`key` (React がリストの要素を見分けるための印)に `project.id` を渡すのは再描画のときにどのカードがどれか React が追跡できるようにするためです。

Day 09: プロジェクト一覧画面を作ろう

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。
- プロジェクトがカード形式で表示されています。

この時点の画面は後で作るページヘッダー(アーカイブ表示の切り替えや新規プロジェクトボタン)がまだ無い状態で、タイトルとカードのグリッドだけが並びます。

スクリーンショット: 下の画像は Step 9 まで書き終えた完成後の画面です。赤枠の中がこの Step で足したカードのグリッドです。いまの自分の画面には右上のアーカイブ表示スイッチと新規プロジェクトボタンがまだ出ていません。どちらも Step 9 で足します。

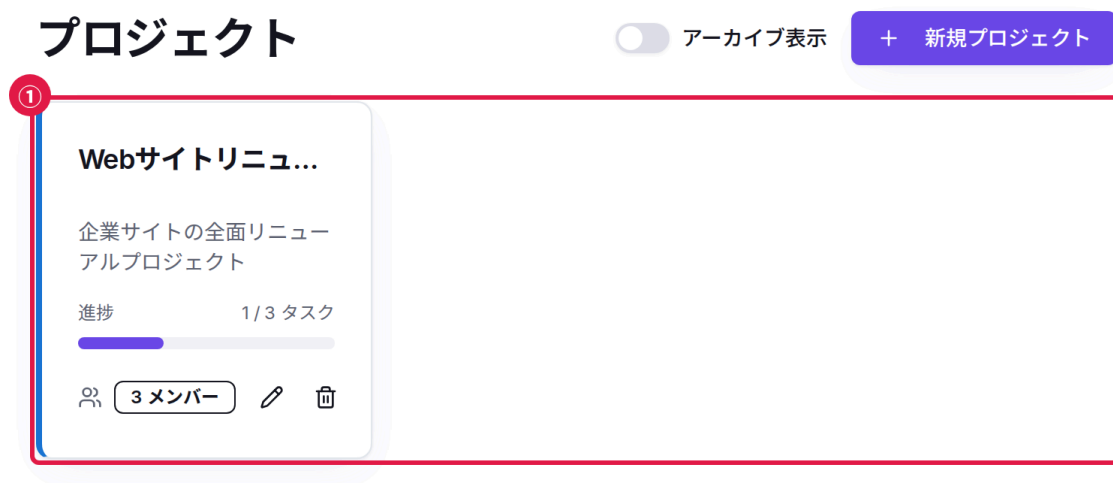


図6: 完成後のプロジェクト一覧。赤枠の中はカードのグリッドで、プロジェクトカードが1枚だけ並ぶ

写っているカードは1枚です。初期データのうち読者がメンバーになっているのは「Webサイトリニューアル」だけなのでこれで正しい状態です。

'DONE' のような文字列リテラルではなく `TASK_STATUS.DONE` 定数を使います。定数を使うとタイプミスを防げて値が変わっても1か所直すだけで済みます。

グリッドの画面幅別列数

画面幅	クラス	列数
スマホ(~640px)	デフォルト (<code>grid-cols-1</code>)	1 列
タブレット(640px~)	<code>sm:grid-cols-2</code>	2 列
PC(1024px~)	<code>lg:grid-cols-3</code>	3 列
ワイド(1280px~)	<code>xl:grid-cols-4</code>	4 列

Step 7 : 空状態のメッセージを追加する(読む目安: 3 分)

ゴール: プロジェクトが0件の場合にメッセージを表示します。

実装:

Step 6 で書いたグリッドの `<div>` の中身を、三項演算子(条件によって2つのうち片方を選ぶ書き方)を使う形に書き直します。Step 6 で書いた `{projects?.map((project) => { から })}` までは先にすべて削除してください。残したまま足すと同じカードの記述が二重になります。

削除したあと、`<div className="grid ...">` と `</div>` の間に、次の3つのブロックを上から順に貼ります。

Day 09: プロジェクト一覧画面を作ろう

```
{/* filepath: src/app/project/page.tsx */}
{/* グリッドdiv内を修正(mapの前に条件分岐追加) */}
{projects && projects.length > 0 ? (
  projects.map((project) => {
    // キャンセル済みは進捗の母数に含めない
    let taskCount = 0;
    let doneCount = 0;
    for (const t of project.tasks ?? []) {
      if (t.status === TASK_STATUS.CANCELLED) continue;
      taskCount++;
      if (t.status === TASK_STATUS.DONE) doneCount++;
    }
    return (
      <ProjectCard
        key={project.id}
        id={project.id}
        name={project.name}
```

`projects && projects.length > 0` と2つ並べているのは確かめたいことが2つあるからです。Step 2 で見たとおり、読み込みが終わるまで `projects` は `undefined` です。手前の `projects &&` が無いとまだ中身のない `undefined` から `length` を読みに行きます。その結果、画面は真っ白になります。後ろの `length > 0` は届いたけれど0件だった場合を切り分けます。

なお Step 3 で読み込み中と最初の取得失敗は手前で `return` しています。ここへ来た時点では `projects` は取得済みなので、0件の表示を通信失敗と混同しません。

ProjectCard の props の途中で切れています。続きを次のブロックで書きます。

```
// filepath: src/app/project/page.tsx
// ProjectCard の続き(props後半)
    description={project.description}
    color={project.color}
    memberCount={
      project.members?.length ?? 0
    }
    taskStats={{
      total: taskCount,
      done: doneCount,
    }}
    onEdit={handleEdit}
    onDelete={handleDelete}
    onClick={handleProjectClick}
    isArchived={project.isArchived}
  />
);
})
```

Day 09: プロジェクト一覧画面を作ろう

props の並びは Step 6 とまったく同じです。書き換えたのは前後の囲みだけです。末尾の `})` が map を閉じます。ただし `? (` で開いた三項演算子はまだ閉じていません。ここで手を止めて保存すると括弧の対応が合わずエラーになります。次のブロックの `) : (` から書き始めて 0 件のときの表示までを続けて入れてください。

```
// filepath: src/app/project/page.tsx
// 三項演算子の else 側(空状態メッセージ)
) : (
  <div className="col-span-full flex
    flex-col items-center justify-center
    py-12 text-center
    text-muted-foreground">
    <p>プロジェクトが見つかりません。</p>
    <p>最初のプロジェクトを作成しましょう！</p>
  </div>
)}
```

確認ポイント:

- ファイルを保存しました。
- プロジェクトが 0 件のときメッセージが表示されます。

0 件の画面を出すにはアカウントを入れ替えます。いま使っている

`admin@example.com` は初期データの「Web サイトリニューアル」のメンバーなので必ず 1 件返るためです。サイドバーからログアウトし、Day 07 の表にある

`empty@example.com` / `password123` でログインし直すとどのプロジェクトにも参加していない状態の画面が出ます。確かめたら `admin@example.com` に戻してください。以降の Step は管理者のまま進めます。

スクリーンショット: 空状態の表示を確認してください。

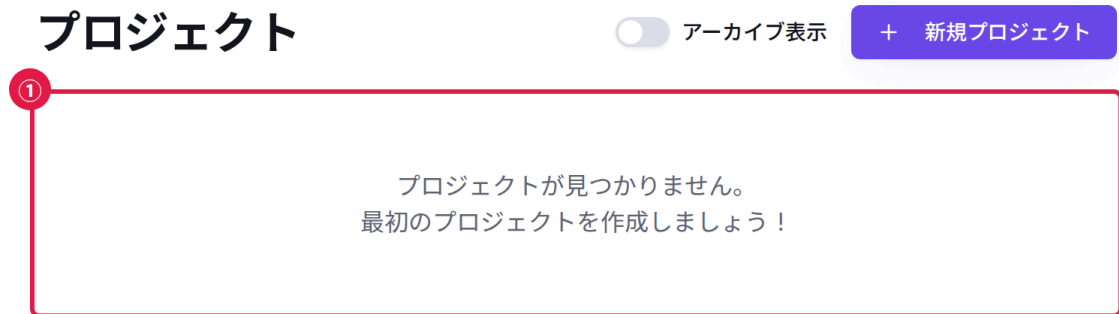


図 7: どのプロジェクトにも参加していないアカウントで開いた一覧。「プロジェクトが見つかりません。」と出ている

`col-span-full` はグリッドの全列にまたがって表示するクラスです。これがないとメッセージが1列分の幅にしか表示されません。

Step 8 : ヘッダー部品のimportと状態を準備する(読む目安: 5分)

ゴール: ボタンとスイッチを表示するための部品と状態を準備します。画面上の配置は Step 9 で行います。

実装:

import 群に以下を追加します。

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx
// import群に追加
import { Button }
  from '@component/ui/button';
import { Switch }
  from '@component/ui/switch';
import { Label }
  from '@component/ui/label';
import { Plus } from 'lucide-react';
```

ここで足す4つはヘッダーに置く操作部品です。`Button` は新規作成ボタン、`Switch` はアーカイブ表示を切り替えるスイッチ、`Label` はそのスイッチの見出し、`Plus` はボタンに添える「+」アイコンです。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。

`ProjectPageContent` 内の `showArchived` を作る `useState` の直後に、ダイアログ用 state とハンドラーを追加します。`if (projectsLoading)` より前に置いてください。

```
// filepath: src/app/project/page.tsx
// ProjectPageContent内に追加
const [dialogOpen, setDialogOpen] =
  useState(false);

const handleCreate = () => {
  setDialogOpen(true);
};
```

`dialogOpen` は新規作成ダイアログを開いているかどうかを覚えておく状態です。
`handleCreate` はボタンが押されたときに呼ばれ、この状態を `true` にします。
ダイアログ本体はまだ無いのでいまは状態が変わるだけです。

確認ポイント:

- ファイルを保存しました。
- `npm run dev` でエラーが出ていません。

ボタンの動作について: 「新規プロジェクト」ボタンをクリックすると

`dialogOpen` が `true` になりますがダイアログ本体は Day 10 で実装します。今日の時点ではボタンを押しても画面に変化はありません。それで正常です。

今日は `npm run lint` に `dialogOpen` の指摘が出ます: `dialogOpen` は値を入れるだけで、読み出すのは Day 10 のダイアログです。今日の時点では「代入しているのに一度も読んでいない変数」に見えるので

`lint/correctness/noUnusedVariables` が出ます。写経の間違いではないのでそのまま進めてください。Day 10 でダイアログを作れば読み出しが増えてこの指摘は消えます。

並ぶ件数は1件とは限りません。Day 05 で断ったとおり、`npm run fix` をまだ実行していなければ整形の差分も一緒に出ます。数ではなく、`dialogOpen` の行があるかどうかを見てください。

`npm run fix` を実行してもこの指摘は直りません。変数名の書き換えで黙らせる修正になるため Biome はこれを「安全でない修正」に分類しており、`--unsafe` を付けたときだけ適用されるからです。`npm run fix` は `--unsafe` を付けていないので `dialogOpen` が勝手に書き換わることはありません。

Step 9 : ページ全体を組み立てる(読む目安: 5分)

ゴール: ヘッダー、グリッド、空状態を組み合わせて `return` を完成させます。

実装:

`ProjectPageContent` の `return` 全体を以下に置き換えます。ここでは Step 6・7 で作ったグリッドと空状態の構造をそのまま使い、ヘッダーと同じ `return` の中へ配置し直します。

```
// filepath: src/app/project/page.tsx
// returnの開始～ヘッダー部分
return (
  <AppLayout>
    <div className="flex flex-col gap-6">
      <div className="flex items-center
        justify-between">
        <h1 className="text-3xl font-bold
          tracking-tight">
          プロジェクト
        </h1>
        <div className="flex items-center
          gap-4">
          <div className="flex
            items-center space-x-2">
            <Switch
              id="show-archived"
              checked={showArchived}
              onCheckedChange={
                setShowArchived} />
            <Label
              htmlFor="show-archived">
              アーカイブ表示
            </Label>
          </div>
        </div>
      </div>
    </div>
  </AppLayout>
)
```

Day 09: プロジェクト一覧画面を作ろう

Step 1 で作った骨組みに、ヘッダーの中身を足していきます。見出しの横に操作エリアを `flex` で横並びにし、そこへアーカイブ表示スイッチを置きます。`checked` と `onCheckedChange` でスイッチを `showArchived` とつなぐと切り替えが一覧の絞り込みに伝わります。

確認ポイント:

- ヘッダーのタイトルとスイッチが表示されています。

続けてボタンとヘッダーの閉じタグを追加します。

```
{/* filepath: src/app/project/page.tsx */}
{/* ヘッダーの続き(ボタンと閉じタグ) */}
    <Button onClick={handleCreate}>
      <Plus
        className="mr-2 h-4 w-4" />
      新規プロジェクト
    </Button>
  </div>
</div>
```

右側に新規作成ボタンを置き、開いていたヘッダーの `<div>` を閉じます。ボタンとスイッチを同じ行にまとめると操作が画面の右上にそろいます。

確認ポイント:

- 「新規プロジェクト」ボタンが右上に表示されています。

再取得に失敗しても前回の一覧が残っている場合は、カードを消さずに注意を出します。ヘッダーの閉じタグとグリッドの間に追加します。

```
{/* filepath: src/app/project/page.tsx */}
{projectsError && projects && !projectAccessDenied && (
  <p role="alert" className="text-sm text-destructive">
    最新の一覧を取得できませんでした。
    前回の一覧を表示しています。
  </p>
)}
```

`projects` が残り、認証・認可の失敗ではない場合だけこの案内を出します。表示する権限が無くなった一覧を、過去のキャッシュから見せないためです。

続けてグリッド部分を追加します。

Day 09: プロジェクト一覧画面を作ろう

```
{/* filepath: src/app/project/page.tsx */}
{/* グリッド開始~map処理 */}
    <div className="grid gap-6
      sm:grid-cols-2 lg:grid-cols-3
      xl:grid-cols-4">
      {projects && projects.length > 0
        ? (projects.map((project) => {
          // キャンセル済みは進捗の母数に含めない
          let taskCount = 0;
          let doneCount = 0;
          for (const t of project.tasks ?? []) {
            if (t.status === TASK_STATUS.CANCELLED)
              continue;

            taskCount++;
            if (t.status === TASK_STATUS.DONE)
              doneCount++;
          }
        })
        : null)}
    </div>
```

ヘッダーの下に、カードを並べるグリッドを置きます。`projects` が空でないときだけ `.map()` に進むよう、三項演算子で分けています。中身は Step 6・7 で書いたものを、完成形の位置に組み直しています。

確認ポイント:

- グリッドコンテナと map 処理が追加されています。

map 内で `ProjectCard` を返します。


```
// filepath: src/app/project/page.tsx
// map内のreturn(カード表示)

    return (
      <ProjectCard
        key={project.id}
        id={project.id}
        name={project.name}
        description={
          project.description}
        color={project.color}
        memberCount={
          project.members?.length
            ?? 0}
        taskStats={{
          total: taskCount,
          done: doneCount }}
        onEdit={handleEdit}
        onDelete={handleDelete}
        onClick={
          handleProjectClick}
        isArchived={
          project.isArchived}
      />);
  })
```

map の中身は Step 6 と同じで、1 件ずつ `ProjectCard` に変換します。ここで書き直すのはヘッダーやグリッドと同じ `return` の中に、正しい入れ子で収めるためです。

Day 09: プロジェクト一覧画面を作ろう

確認ポイント:

- ProjectCardのpropsが正しく渡されています。

続けて空状態の表示を追加します。

```
// filepath: src/app/project/page.tsx
// 空状態の分岐(上の続き)
    ) : (
      <div className="col-span-full
        flex flex-col items-center
        justify-center py-12
        text-center
        text-muted-foreground">
```

三項演算子の else 側はプロジェクトが0件のときに表示する入れ物です。

`col-span-full` でグリッドの全幅を使い、メッセージを中央に寄せます。

確認ポイント:

- 三項演算子の else 側が追加されています。

最後に空状態のメッセージと全体の閉じタグです。

```

{/* filepath: src/app/project/page.tsx */}
{/* 空メッセージ～全体の閉じタグ */}
    <p>{showArchived ?
'アーカイブ済みのプロジェクトはありません。'
      : '進行中のプロジェクトはありません。'}</p>
    </div>
  )}
</div>
</div>
</AppLayout>
);

```

最後に空状態のメッセージを入れ、開いていた `<div>` と `AppLayout` を順に閉じます。これで `return` が完成し、ヘッダー・一覧・空状態が1つの画面にそろいます。

空状態は `showArchived` に合わせて「進行中」と「アーカイブ済み」を言い分けます。アーカイブ済みが0件のときに、新規作成が必要だと誤解させないためです。

確認ポイント:

- `npm run dev` でエラーが出ていません。
- 「新規プロジェクト」ボタンが右上に表示されています。
- アーカイブ表示スイッチを押すと一覧が空になります(後述)。

スクリーンショット: ヘッダー付きの完成画面の表示を確認してください。



図 8: 右上に「新規プロジェクト」ボタンとアーカイブ表示スイッチが並んだ一覧画面

Step 10 : 動作確認(読む目安: 3 分)

ゴール: プロジェクト一覧の全機能を確認します。

開発サーバーを起動して確認します。

```
# filepath: ターミナル
# 開発サーバーを起動
npm run dev
```

確認ポイント:

- ターミナルの `Local:` に表示された URL の `/project` を開けます。

以下の項目を順番に確認してください。

#	確認内容	期待される動作
1	<code>/project</code> にアクセス	カードが表示される
2	ブラウザ幅を変える	1枚のカードの幅が段階的に変わる
3	「新規プロジェクト」ボタン	右上に表示されている(※)
4	カードの内容	色帯・メンバー数・進捗がある
5	ページ読み込み時	通信待ちの間はスピナーが表示される
6	表示対象のプロジェクトが0件の場合	スイッチがオフなら「進行中のプロジェクトはありません。」、オンなら「アーカイブ済みのプロジェクトはありません。」と表示される

※「新規プロジェクト」ボタン・カードの「編集」「削除」ボタン・カードクリック時の遷移はDay 10-12 で本実装します。今日の時点ではクリックしても何も起きませんが正常な動作です。

Day 09: プロジェクト一覧画面を作ろう

スクリーンショット: ブラウザの幅を 430px まで縮めたときの姿です。サイドバーが隠れ、見出しと操作部品は横1列のまま幅だけが詰まります。今日書いた

`flex items-center justify-between` には折り返しの指定が無いので見出しが2行になり、スイッチのラベルが縦書きのように潰れます。狭い画面で縦に積み直す形は Day 27 で入れます。



図9: 幅を狭めたプロジェクト一覧。サイドバーが消え、2行に折り返した見出しの右にアーカイブ表示スイッチと新規プロジェクトボタンが横1列で詰まって並んでいる

カードが1枚しか無いので列数そのものは見えませんが、カードの幅は画面幅に応じて変わります。Day 10 以降でカードを増やすと2列・3列・4列の並びも確認できます。

手元のDBからすぐ返事が届くと、スピナーは目で追えないほど短時間で消えます。見えなくても異常ではありません。

確認ポイント:

- カードがグリッドで並んでいます。
- 返事を待つ間はローディングスピナーを出す分岐があります。
- 画面幅を変えると1枚のカードの幅が変わります。
- アーカイブ表示スイッチを ON にすると一覧が空になり、OFF に戻すと元のカードが並びます。

このスイッチは「アーカイブ済みのプロジェクトだけ」を表示します。追加で表示するものではありません。いまはアーカイブ済みが1件も無いので ON にすると一覧が空になります。これで正常です。アーカイブする操作そのものは Day 11 で作ります。

Pro パターンで書こう(プロジェクト一覧は `useQuery` に任せる)

データ・読み込み中・エラーの3状態を `useQuery` にまとめると状態の同期漏れが起きなくなります。なぜ直前の1文の書き方をするのか、**Before/After** で見比べてみましょう。

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(参考・実ファイルには対応しません)

import { useEffect, useState } from 'react';

export function ProjectListPanel() {
  const [projects, setProjects] = useState([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    fetch('/api/projects')
      .then((res) => res.json())
      .then(setProjects)
      .finally(() => setIsLoading(false));
  }, []);

  return isLoading ? <p>読み込み中</p> : <ProjectList />;
}
```

このコードも一覧は表示できます。困るのは状態を自分で数え続けなければならない点です。`fetch` が失敗したときの分岐がどこにも無いので通信が切れても `isLoading` は `false` になり、画面には空の一覧が出ます。読者には「0件でした」としか見えません。

`useEffect` の依存配列が空なので取得は最初の1回だけです。今日のアーカイブ表示スイッチのように条件が増えた時点で、この形は作り直しになります。

このコードの問題点:

- `projects` と `isLoading` を自分で更新し、失敗した場合の状態も別に用意しなければなりません。
- キャッシュや再取得の仕組みを毎回考えることになり、一覧画面ごとに実装がばらつきます。
- `res.json()` の結果を検証していないためAPI が想定外の形を返してもそのまま受け取ってしまいます。

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(参考・実ファイルには対応しません)

import { api } from '@trpc/react';

export function ProjectListPanel() {

  const {
    data: projects = [],
    isLoading,
    error,
  } = api.project.getAll.useQuery({
    isArchived: false,
  });

  if (isLoading) {
    return <p>読み込み中</p>;
  }

  if (error) {
    return <p>{error.message}</p>;
  }

  return <ProjectList projects={projects} />;
}
```

`data: projects = []` はデータが未取得の間に使う既定値です。`isLoading` や `error` はこの指定とは独立しています。先にその2つで表示を分ければ既定値を使っても読み込み中や取得失敗を0件と混同せずに済みます。

取得・読み込み中・失敗の3つが1つのフックから揃って返ります。`isLoading` と `error` を先に分岐で片付けているのでその後ろでは `projects` の中身だけを考えれば済みます。`data: projects = []` と既定値を置いているのはこの位置から先で「配列である」と決め切るためです。分岐を通り抜けるたびに `undefined` かどうかを確かめる書き方は確かめ忘れた1か所で画面が真っ白になります。

このアプリは tRPC の変換に `superjson`(日付などを形を保ったまま送り届けるライブラリ)を使っています。`createdAt` のような日付は `Date` のまま届くので `new Date()` で組み立て直す処理は要りません。

このコードの強み:

- `data`、`isLoading`、`error` が1つのフックから揃って返るので状態管理がまとまります。
- tRPCとTanStack Queryがキャッシュ、再取得、型推論(型を書かなくてもTypeScript が自動で型を割り出してくれる仕組み)をまとめて面倒見てくれます。
- `project.name` や `project.description` の型がサーバー側ルーターからつながるので変更に強いです。

覚えておきたいエッセンス

一覧取得は `useEffect` (画面の表示が終わった後に処理を実行する React の機能)で手作りするよりデータ取得専用のフックに任せる ほうが安定します。tRPCの `useQuery` は取得・状態・型をまとめて引き受けてくれるのです。

完成コード全体

今日は4つのファイルに触りました。ページ側は Step 1 から Step 9 まで少しずつ足していったので途中でどこへ貼ったか分からなくなった場合は以下のコードと自分のファイルを見比べてください。上から順に読めば断片がどう1つのファイルになったかを確認られます。

ファイル	役割	対応する Step
<code>src/server/api/routers/project.ts</code>	プロジェクト一覧 を返す手続き	Step 0
<code>src/server/api/root.ts</code>	手続きの一覧表	Step 0
<code>src/app/project/page.tsx</code>	プロジェクト一覧 画面	Step 1 ~ Step 9
<code>src/component/layout/app-layout.tsx</code>	サイドバーのプロ ジェクト導線	Step 1

`src/component/layout/app-layout.tsx`

ここは Step 1 で変更した2か所だけを掲載します。ほかの行は Day 08 で書いた状態を残してください。

```
// filepath: src/component/layout/app-layout.tsx
// 完成版: アイコンのインポート
import { FolderOpen, LayoutDashboard, Logout } from
  'lucide-react';
```

`FolderOpen` が今日追加した名前です。残りの2つはダッシュボード項目とログアウトボタンが使うため、プロジェクト導線を足したあとも削除しません。

```
// filepath: src/component/layout/app-layout.tsx
// 完成版: サイドバーのメニュー項目
const menuItems: MenuItem[] = [
  {
    text: 'ダッシュボード',
    icon: <LayoutDashboard className="h-5 w-5" />,
    path: '/dashboard',
  },
  {
    text: 'プロジェクト',
    icon: <FolderOpen className="h-5 w-5" />,
    path: '/project',
  },
];
```

Day 09 で作った `/project` と同じ日にリンクを追加したので、サイドバーに見える2項目はどちらも開けます。

`src/server/api/routers/project.ts`

取り込み:

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/server/api/routers/project.ts
// 完成版: 取り込み
import type { Prisma } from '@prisma/client';
import { TRPCError } from '@trpc/server';
import { z } from 'zod';
import { USER_ROLE } from '@lib/constant/roles';
import { prisma } from '@lib/prisma';
import { createTRPCRouter, protectedProcedure } from
'../trpc';
import { USER_SELECT } from '../helpers/select';
```

`import type` で始まる1行目だけ書き方が違うのは `Prisma` を値としてではなく型の注釈にしか使わないからです。この書き方にしておくとならぬコードから消えます。並び順は `npm run fix` で Biome(このプロジェクトのコード整形ツール)が決めるので自分が書いた順番と違っていても手で直す必要はありません。

手続きの骨組みと入力:

```
// filepath: src/server/api/routers/project.ts(同じファイルの続き)
// 完成版: 手続きの骨組みと入力
export const projectRouter = createTRPCRouter({
  getAll: protectedProcedure
    .input(
      z
        .object({
          userId: z.string().cuid().optional(),
          isArchived: z.boolean().optional(),
        })
        .optional(),
    )
    .query(async ({ ctx, input }) => {
      const where: Prisma.ProjectWhereInput = {};

```

`protectedProcedure` から書き始めているのでログインしていない人はこの先へ進めません。認証の確認をここに置いておけば手続きを増やすたびに同じ判定を書かずに済みます。`.optional()` が入れ子で2か所に付いているのは項目だけ省く呼び方と、条件そのものを渡さない呼び方の両方を許すためです。

他人の一覧を守る判定:

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/server/api/routers/project.ts(同じファイルの続き)
// 完成版: 他人の一覧を守る判定

    if (input?.userId && input.userId !==
ctx.session.userId) {

        if (ctx.session.role !== USER_ROLE.ADMIN) {

            throw new TRPCError({

                code: 'FORBIDDEN',

                message: '管理者権限が必要です',

            });

        }

    }

}
```

この判定を検索条件の組み立てより前に置いてあるのが要点です。後ろへ動かすと弾かれるはずの呼び出しでも問い合わせが1回走ります。`throw` は処理をその場で打ち切る命令なのでここを通れなかった呼び出しは以降の行に届きません。

検索条件の組み立て:


```
// filepath: src/server/api/routers/project.ts(同じファイルの続き)
// 完成版: 検索条件の組み立て

    if (!input?.userId) {
      where.members = {
        some: { userId: ctx.session.userId },
      };
    } else {
      where.members = {
        some: { userId: input.userId },
      };
    }

    if (input?.isArchived !== undefined) {
      where.isArchived = input.isArchived;
    }
  }
}
```

`where` を空の入れ物として用意しておき、決まった条件だけを足していく形にしています。条件の組み合わせが増えても `findMany` の呼び出しは1つのままで済みます。`isArchived` を `!== undefined` で見ているのは `false` を渡された場合と、渡されなかった場合を分けるためです。`if (input?.isArchived)` と書くと `false` は未指定と区別できません。

取得と返却:

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/server/api/routers/project.ts(同じファイルの続き)
// 完成版: 取得と返却

return await prisma.project.findMany({
  where,
  include: {
    members: {
      include: {
        user: {
          select: USER_SELECT,
        },
      },
    },
    tasks: {
      select: {
        id: true,
        status: true,
      },
    },
  },
  orderBy: { createdAt: 'desc' },
});
```

メンバーとタスクを一緒に取っておくと画面側は追加の通信なしで人数と進捗を出せます。いちばん内側で `select` を使い、返す項目を絞っているのはパスワードのような返してはいけない値を通さないためです。末尾の `});` が `getAll` を、`});` が `projectRouter` 全体を閉じます。ここが1つでも足りないと開発サーバーは起動の時点で止まります。

`src/server/api/root.ts`

手続きの一覧表:

```
// filepath: src/server/api/root.ts
// 完成版: 手続きの一覧表
import { authRouter } from './routers/auth';
import { projectRouter } from './routers/project';
import { createCallerFactory, createTRPCRouter } from
'./trpc';

export const appRouter = createTRPCRouter({
  auth: authRouter,
  project: projectRouter,
});

export type AppRouter = typeof appRouter;

export const createCaller = createCallerFactory(
appRouter);
```

Day 09: プロジェクト一覧画面を作ろう

ここに並べた名前がそのまま画面側の呼び名になります。

`project: projectRouter` と書いたので `api.project.getAll` が使えます。左側の名前を `projects` にすると画面側も `api.projects.getAll` に変わります。`export type AppRouter` の行があるおかげで、画面側は関数名も引数の形もサーバー側の定義から受け取れます。

`src/app/project/page.tsx`

取り込み:

```
// filepath: src/app/project/page.tsx
// 完成版: 取り込み
'use client';

import { Plus } from 'lucide-react';
import { Suspense, useState } from 'react';
import { AppLayout } from '@component/layout/app-layout';
import { ProjectCard } from
  '@component/project/project-card';
import { Button } from '@component/ui/button';
import { Label } from '@component/ui/label';
import { PageLoadingSpinner } from
  '@component/ui/loading-spinner';
import { Switch } from '@component/ui/switch';
import { TASK_STATUS } from '@lib/constant/status';
import { api } from '@trpc/react';
```

Step 1・Step 2・Step 4・Step 8 で少しずつ足した取り込みが最後にはこの10行に落ち着きます。`react` からの取り込みが1行にまとまっているのは同じ取り寄せ元を2行に分けても意味が変わらないからです。`'use client'` はファイルの1行目に置きます。取り込みの後ろへ動かすと `useState` の行でエラーになります。

状態とサーバーへの問い合わせ:

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 状態とサーバーへの問い合わせ

function ProjectPageContent() {
  const [showArchived, setShowArchived] = useState(false);
  const [dialogOpen, setDialogOpen] = useState(false);

  const {
    data: projects,
    isLoading: projectsLoading,
    error: projectsError,
  } = api.project.getAll.useQuery({
    isArchived: showArchived,
  });
}
```

`showArchived` を `useQuery` の引数に渡してあるのでスイッチを切り替えるだけで一覧が取り直されます。取り直す関数を自分で呼ぶ書き方にすると呼び忘れた場所だけ古い一覧が残ります。`data: projects` と名前を付け替えているのはこの画面に問い合わせが増えたときにどの `data` なのかを見分けられるようにするためです。

ボタンの受け皿:

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: ボタンの受け皿

const handleEdit = (projectId: string) => {
  void projectId;
};

const handleDelete = (projectId: string) => {
  void projectId;
};

const handleProjectClick = (id: string) => {
  void id;
};

const handleCreate = () => {
  setDialogOpen(true);
};
```

3つの受け皿が何もしないのは行き先の画面をまだ作っていないからです。中身が空でも用意しておくカードに渡す値の形が今日のうちに決まります。

認証・認可の判定と読み込み中の表示:

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 認証・認可の判定と読み込み中の表示

const projectAccessDenied =
  projectsError?.data?.code === 'UNAUTHORIZED' ||
  projectsError?.data?.code === 'FORBIDDEN';

if (projectsLoading) {
  return (
    <AppLayout>
      <PageLoadingSpinner />
    </AppLayout>
  );
}
```

`if (projectsLoading)` を `return` より前に置いたなのでこの行より下では読み込み中の場合を考えずに済みます。

最初の取得失敗を表示する部分:

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 最初の取得失敗

if (projectsError && (!projects ||
projectAccessDenied)) {
  return (
    <AppLayout>
      <p role="alert">
        {projectAccessDenied
          ? 'この一覧を表示する権限がありません。'
          : 'プロジェクトを読み込めませんでした。'}
      </p>
    </AppLayout>
  );
}
```

最初の取得に失敗した場合は、0件のメッセージを出さず取得失敗を表示します。認証・認可の失敗なら、前回の `projects` が残っていてもカードを表示しません。それ以外の再取得失敗では、取得済みのカードを表示し続けます。

見出しの部分:


```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 見出しまで

return (
  <AppLayout>
    <div className="flex flex-col gap-6">
      <div className="flex items-center
justify-between">
        <h1 className="text-3xl font-bold
tracking-tight">
          プロジェクト
        </h1>

```

AppLayout がいちばん外側にあるのでこのページにもサイドバーと認証の判定が付きます。ページ側で認証を確かめる行が1つも無いのはその仕事を囲みへ預けたからです。**justify-between** を付けた **<div>** が見出しと操作部品を左右の端へ振り分けます。

スイッチとボタン:

```

    {/* filepath: src/app/project/page.tsx(同じファイルの続き) */}
    {/* 完成版: スイッチとボタン */}
    <div className="flex items-center gap-4">
      <div className="flex items-center space-x-2">
        <Switch
          id="show-archived"
          checked={showArchived}
          onCheckedChange={setShowArchived} />

```

```
        <Label htmlFor="show-archived">
            アーカイブ表示
        </Label>
    </div>

    <Button onClick={handleCreate}>
        <Plus className="mr-2 h-4 w-4" />
        新規プロジェクト
    </Button>
</div>
</div>

{projectsError && projects
&& !projectAccessDenied && (
    <p role="alert" className="text-sm
text-destructive">
        最新の一覧を取得できませんでした。
        前回の一覧を表示しています。
    </p>
)}
```

`Switch` の `id` と `Label` の `htmlFor` に、同じ `show-archived` を書いてあります。この2つが揃っていると文字のほうを押してもスイッチが切り替わります。片方だけ書き換えると見た目は変わらないまま、文字を押しても反応しない状態になります。`onCheckedChange` に `setShowArchived` を直接渡しているので切り替えた値がそのまま問い合わせの条件になります。その後ろの `projectsError` の分岐は、認証・認可以外の再取得が失敗したときだけ、取得済みの一覧と注意を一緒に表示します。

カードを並べる枠:

```

    {/* filepath: src/app/project/page.tsx(同じファイ
ルの続き) */}
    {/* 完成版: カードを並べる枠 */}
    <div className="grid gap-6 sm:grid-cols-2
lg:grid-cols-3 xl:grid-cols-4">
      {projects && projects.length > 0
        ? (projects.map((project) => {
          // キャンセル済みは進捗の母数に含めない
          let taskCount = 0;
          let doneCount = 0;
          for (const t of project.tasks ?? []) {
            if (t.status === TASK_STATUS.CANCELLED)
continue;

            taskCount++;
            if (t.status === TASK_STATUS.DONE)
doneCount++;
          }
        })
        : null)}
    </div>

```

`projects &&` を手前に置いてあるのは届く前の `projects` から `length` を読むと画面が真っ白になるからです。件数を数えるループを `map` の中に置いているのでプロジェクトごとに自分のタスクだけを数えます。完了の判定に `TASK_STATUS.DONE` を使うと打ち間違えたときに画面を開く前に気づけます。

1件ぶんのカード:

Day 09: プロジェクト一覧画面を作ろう

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 1件ぶんのカード

    return (
      <ProjectCard
        key={project.id}
        id={project.id}
        name={project.name}
        description={project.description}
        color={project.color}
        memberCount={project.members?.length ??
0}

        taskStats={{
          total: taskCount,
          done: doneCount,
        }}
        onEdit={handleEdit}
        onDelete={handleDelete}
        onClick={handleProjectClick}
        isArchived={project.isArchived}
      />);
    )
```

渡している値のうち `name` から `isArchived` までは Step 0 の `getAll` が返してきたものです。`memberCount` と `taskStats` だけ、この場で数えた値を渡します。`onClick` があるので、カード全体をマウスで押す操作に加え、プロジェクト名を `Tab` キーで選んで `Enter` キーでも開けます。`key` に `project.id` を使うのは並びが変わったときにどのカードが動いたかを React へ伝えるためです。手元

で `key` を消すと表示自体は出ますがブラウザの開発者ツールのコンソールに警告が残ります。この一覧はデータが届いてからブラウザ側で組み立てるので警告もブラウザ側に出ます。

0件のときの表示と閉じタグ:

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 0件のときの表示と閉じタグ
    ) : (
      <div className="col-span-full flex flex-col
items-center justify-center py-12 text-center
text-muted-foreground">
        <p>{showArchived ?
'アーカイブ済みのプロジェクトはありません。'
:
'進行中のプロジェクトはありません。'}</p>
      </div>
    )}
  </div>
</div>
</AppLayout>
);
}
```

`col-span-full` を付けているのはこの枠を格子の1列ぶんの幅で終わらせないためです。付け忘れるとメッセージが左端の細い列だけに押し込まれます。閉じタグは内側から順に、格子・縦並びの入れ物・`AppLayout` と続きます。手元で括弧が合わないときは `) : (` と `)}` の2行の位置を先に確かめてください。

Day 09: プロジェクト一覧画面を作ろう

画面として書き出す部分:

```
// filepath: src/app/project/page.tsx(同じファイルの続き)
// 完成版: 画面として書き出す部分
export default function ProjectPage() {
  return (
    <Suspense
      fallback={<PageLoadingSpinner />}>
    <ProjectPageContent />
    </Suspense>
  );
}
```

画面として読まれるのはこちらで、本体は `ProjectPageContent` に分けてあります。分けておくと準備が終わるまでの仮表示を `Suspense` の1か所で受け止められます。`export default` を付けた関数がページの入口になるのでこの行を本体の側へ移すと囲みが働かないまま本体だけが表示されます。

完成コードを書き終えたらターミナルで `npm run type-check` を実行してください。型のエラーが出た場合は表示されたファイル名と行番号を確認して直します。画面の動作は本文の操作手順で別に確かめます。

今日のまとめ

- ☐ `useQuery` でサーバーからデータを取得できました。
- ☐ グリッドレイアウトでカード一覧を表示できました。
- ☐ 読み込み中・取得失敗・空状態を分けて表示できました。
- ☐ 新規プロジェクトボタンの準備ができました。

つまずきポイント

`projects` の型が `any` になる / 型補完が効かない

原因

Prisma のクライアント型が生成されていません。

解決方法

開発サーバーを動かしているターミナルで `Ctrl + C` を押して停止します。同じプロジェクトのフォルダで `npx prisma generate` を実行し、終了後に `npm run dev` で開発サーバーをもう一度起動してください。

データが表示されない

原因

API が呼ばれていません。

解決方法

`useQuery()` の呼び出しを確認します。

カードが縦一列になる

原因

グリッドクラスの指定が漏れています。

解決方法

`sm:grid-cols-2 lg:grid-cols-3` を確認します。

Day 09: プロジェクト一覧画面を作ろう

TypeScript の型エラー

原因

ハンドラーの型が一致していません。

解決方法

`(id: string) => void` になっているか確認します。

`PageLoadingSpinner` が見つからない

原因

import パスが間違っています。

解決方法

`@/component/ui/loading-spinner` を確認します。

`TASK_STATUS` が見つからない

原因

import パスが間違っています。

解決方法

`@/lib/constant/status` を確認します。

サイドバーが二重に表示される

原因

同時に描画される2箇所へ `AppLayout` を置いています。

解決方法

`ProjectPage` の `return` を `AppLayout` で囲んでいないか確認します。

`ProjectPageContent` がすでに囲んでいるため、その外側でも囲むと二重になります。

読み込み中だけサイドバーが消える

原因

`ProjectPageContent` の `if (projectsLoading)` が `AppLayout` で囲まれていません。

解決方法

この分岐のスピナーも `AppLayout` で囲みます。`Suspense` の `fallback` は本体と入れ替わりで表示されるため、そちらは囲まなくても二重にはなりません。

今日学んだ用語

用語	意味
<code>useQuery</code>	tRPC/React Query のデータ取得フック
<code>Suspense</code>	子コンポーネントの準備中に代替のUIを表示するReactの仕組み
グリッドレイアウト	CSS Grid で要素を格子状に配置する仕組み

用語	意味
レスポンシブ	画面幅に応じてレイアウトを変えるデザイン手法

応用課題: 最初に表示する絞り込みを変える

スイッチを押す前の状態から問い合わせの条件を変えましょう。画面の初期値が一覧取得にも使われることを確かめます。

(1) `src/app/project/page.tsx` の `showArchived` を作る
`useState(false)` を `useState(true)` に変えて保存します。`dialogOpen` の初期値は変えません。

(2) `/project` を再読み込みします。アーカイブ表示がONになり、アーカイブ済みのプロジェクトだけが対象になります。初期データのままなら空状態が出ます。

(3) スイッチをOFFにします。元の未アーカイブの一覧へ戻れば成功です。最初からOFFなら `useState(true)` に変えた箇所と保存・再読み込みを確認します。

(4) 初期値を `false` へ戻して保存し、再読み込み後にスイッチがOFFで始まることを確認します。

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `getAll` の

`where.members = { some: { userId: ctx.session.userId } }` はどのプロジェクトを選ぶ条件ですか。

A. ログイン中の利用者がメンバーに含まれるプロジェクトだけを選びます。`some` は「関係する行のうち少なくとも1件が条件に合う」という意味です。メンバーの行を1件ずつ見て自分の `userId` が見つかったプロジェクトだけが残ります。この条件が無いと他人のプロジェクトまで一覧に並びます。

Q2. `const { data: projects } = ...` を `const { data: projects = [] } = ...` に書き換えると画面はどう変わりますか。

A. 今日の完成コードでは画面の見た目は変わりません。未取得の `projects` が空配列になりますが `projectsLoading` による読み込み中の表示はそのまま働きます。読み込み中と0件を区別するには配列の長さだけでなく、この読み込み状態を使います。

Q3. 空状態を出す `<div>` に `col-span-full` を付けるのはなぜですか。

A. この `<div>` がグリッドの子だからです。付けないとグリッドは他のカードと同じ1列分の幅を割り当てます。案内の文章が細い列に押し込まれ、左端に寄って表示されます。`col-span-full` は「全部の列にまたがれ」という指定で、これで横いっぱいになります。

次回予告

Day 10 ではプロジェクトの新規作成機能を実装します。ダイアログ(モーダル)を使ったフォーム入力と、tRPC の `useMutation` でデータを保存する方法を学びます。

Day 09 完成形コード(参照用)

`src/app/project/page.tsx` (完成版との違いの説明)

完成版には `src/app/project/page.tsx` がありますがそちらは Day 12 と Day 27 まで書き足した後の姿です。作成ダイアログ、削除確認、アーカイブ、詳細表示が入っているので今日の終わりの手元のコードより長くなります。今日の時点では一覧を取ってきて並べるところまでが書けていれば大丈夫です。足りない部分はこれから何日かけて自分で足していきます。(販売用 ZIP に完成版の `src/` は入っていません。ここに書いた完成版との差はこの先どう伸びるかの説明として読んでください。合否は教材内の確認ポイントで判断します)。

`src/server/api/root.ts`

Day 09 を終えた時点の `root.ts` は Day 07 で登録した `auth` と、今日 Step 0-6 で登録した `project` の2つだけです。`comment` や `task` などとはそれを使う Day で1つずつ足していくので今はまだ登場しません。

```
// filepath: src/server/api/root.ts
import { authRouter } from './routers/auth';
import { projectRouter } from './routers/project';
import { createCallerFactory, createTRPCRouter } from
'./trpc';

export const appRouter = createTRPCRouter({
  auth: authRouter,
  project: projectRouter,
});

export type AppRouter = typeof appRouter;

export const createCaller = createCallerFactory(
appRouter);
```

手元の `root.ts` がこの形になっていれば Step 0-6 の登録は正しく終わっています。ここに並べた名前がそのまま画面側の呼び名です。

`project: projectRouter` の行を消すと `api.project.getAll` は呼べなくなります。一覧にはカードが1枚も出ません。Day 10 以降はこの `createTRPCRouter({ ... })` の中へ1行ずつ足していきます。

次に読むもの

- 前の日: Day 08^{*1}

1. https://drive.google.com/file/d/1Oj1x_743DXVEeRUK7Sgl_dRZ9LGe6O6Y/view?usp=drivesdk

Day 09: プロジェクト一覧画面を作ろう

- 次の日: Day 10^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}
- 言葉の意味: 用語集^{*6}

奥付

タイトル	Day 09: プロジェクト一覧画面を作ろう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。

-
2. https://drive.google.com/file/d/1XswFwEDBx3muA4VnfHQz_Zd4YjrJVTRw/view?usp=drivesdk
 3. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk
 4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
 5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>
 6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>